SCIF30005 Forest Fire Mini Project

Student ID: *[2225197]*, Student Name: *[Zhaoyang Yu]*

February 24, 2026

Abstract

This report presents a parallel implementation of a forest fire cellular automaton in C++ with MPI and analyses its statistical behaviour and parallel performance. The model is simulated on square grids of size $N \times N$ with initial tree occupation probability p, where trees in the top row are ignited and fire spreads using a von Neumann neighbourhood. Three main questions are studied: (i) how the probability of the fire reaching the bottom depends on p, (ii) how many repeat runs M are needed for convergence of averaged quantities, and (iii) how the MPI implementation scales with the number of tasks on BluePebble.

The results show a sharp transition in the probability of reaching the bottom around $p \approx 0.59$, which becomes steeper for larger grid size ($N = 500$ compared to $N = 100$), consistent with finite-size effects. The average burn duration also peaks near the transition region, indicating longer-lived fires when large connected clusters form. Convergence analysis shows that $M = 50$ is often sufficient for qualitative conclusions, but larger M (typically 100–200 near the transition) is needed for tighter confidence intervals. Strong-scaling results show substantially better speedup for $N = 500$ than for $N = 100$, with performance limited at high task counts by halo communication and per-step collective synchronisation.

1 Introduction

The forest fire model is a simple cellular automaton that exhibits threshold-like behaviour and is often used as an accessible example of how local rules can produce complex global dynamics. In this project, a square grid is populated randomly with trees, a fire is ignited at the top, and the simulation evolves in discrete time until no cells are burning. The key observable of interest is whether the fire reaches the bottom row, which behaves similarly to a percolation-style spanning event.

This project has two scientific aims and one computational aim. First, the dependence of the model outcomes on the tree density p is studied, including the probability that the fire reaches the bottom and the average number of time steps before the fire extinguishes. Second, convergence with respect to the number of repeat runs M is investigated, with particular attention to values of p near the transition region where fluctuations are largest. Third, the model is parallelised using MPI and tested on BluePebble to analyse strong-scaling performance for two grid sizes ($N = 100$ and $N = 500$).

The report is organised as follows. Section 2 describes the model, implementation, and experimental setup. Section 3 presents the scientific results and the performance analysis. Section 4 discusses limitations and possible improvements, and Section 6 summarises the main conclusions.

2 Methods

2.1 Forest fire model

The simulation uses a square grid of size $N \times N$. Each cell is either empty, a living tree, burning, or dead. The initial condition is generated by independently placing a tree in each cell with probability p. At time $t = 0$, all living trees in the top row are set on fire.

The model then evolves in discrete timesteps according to the following rules:

- A living tree ignites if at least one of its von Neumann neighbours (up, down, left, right) is burning.
- A burning tree becomes dead in the next timestep.
- Empty cells remain empty and dead cells remain dead.

The simulation terminates when no cells are burning. For each run, two outputs are recorded:

1. the number of timesteps until extinction;
2. whether the fire reached the bottom row at any point.

2.2 MPI parallel implementation

The model is implemented in C++ and parallelised using MPI. A one-dimensional domain decomposition is used, where the grid is split into contiguous blocks of rows across MPI tasks. Each task stores its local rows together with ghost (halo) rows to represent neighbouring data required for updates near subdomain boundaries.

At each timestep, neighbouring tasks exchange boundary rows using blocking `MPI_Sendrecv` calls. This updates the halo rows before local cell states are advanced. Two collective reductions are also performed per timestep:

- a global reduction to determine whether any task still has burning cells (termination condition);
- a global reduction to determine whether the fire has reached the bottom row.

This design is straightforward and correct, but introduces synchronisation overhead that becomes more significant at high task counts and small problem sizes.

2.3 Statistical estimates and uncertainty

For each parameter set (N, p), the simulation is repeated for M independent runs using different random seeds. The estimated probability of reaching the bottom is computed as the sample mean of a Bernoulli indicator:

$$\hat{P}(\text{reach bottom}) = \frac{1}{M} \sum_{m=1}^{M} I_m.$$

The average number of steps is also recorded. Error bars in the plots represent 95% confidence intervals (CI95) estimated from repeated independent runs / batch summaries generated in the analysis pipeline.

2.4 Experimental design

Three groups of experiments were performed:

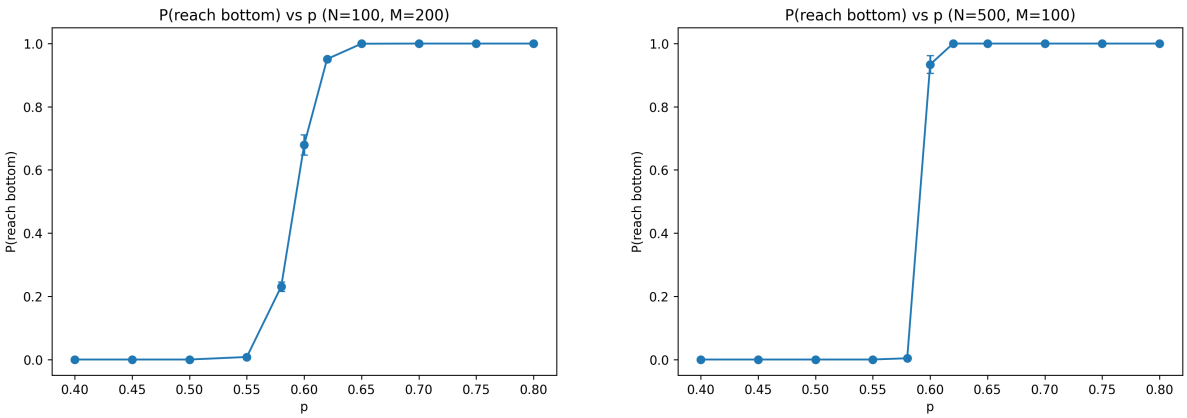
1. **Behaviour versus p:** parameter sweeps over $p \in [0.4, 0.8]$ for $N = 100$ and $N = 500$, measuring $\hat{P}$(reach bottom) and average steps.
2. **Convergence with respect to M:** runs near the transition region ($p = 0.58, 0.60, 0.62$) for increasing M to assess stability and uncertainty.
3. **Performance analysis:** strong-scaling tests at fixed $p = 0.6$ using multiple MPI task counts for $N = 100$ and $N = 500$.

3 Results

3.1 Transition in the probability of reaching the bottom

Figures 1a and 1b show the estimated probability that the fire reaches the bottom row as a function of tree density p for $N = 100$ and $N = 500$, respectively. In both cases, a sharp transition is observed around $p \approx 0.59$. For low p, the fire usually dies out before spanning the grid, whereas for sufficiently large p, a connected path of trees is almost always present and the fire reaches the bottom with probability close to 1.

The transition becomes noticeably steeper for $N = 500$, which is consistent with finite-size effects: larger grids reduce smoothing due to limited system size and produce a more step-like transition. This behaviour supports the interpretation of the model as exhibiting percolation-like threshold behaviour. A practical estimate of the threshold can be taken from the approximate 50% crossing, which lies near $p \approx 0.59$ for both grid sizes.



(a) $N = 100$, $M = 200$.

(b) $N = 500$, $M = 100$.

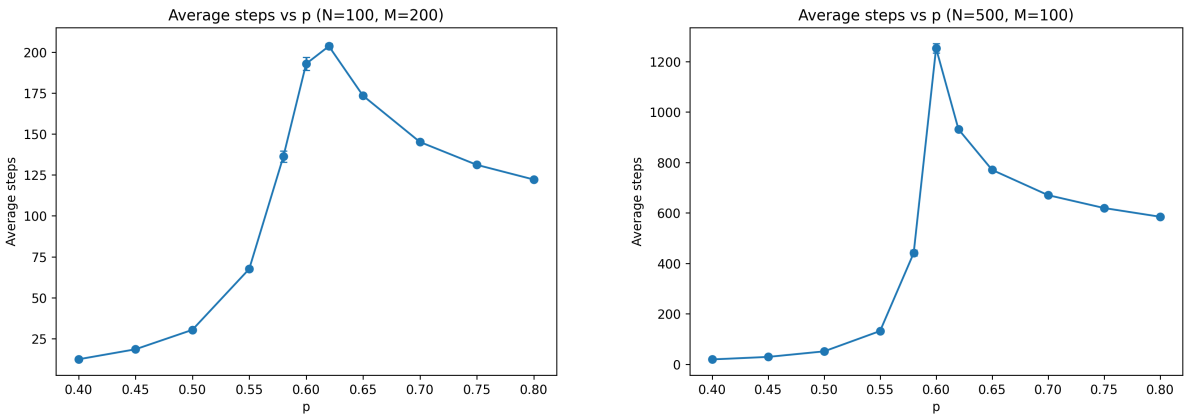
Figure 1: Estimated probability that the fire reaches the bottom row as a function of tree density p. Error bars show 95% confidence intervals. The transition sharpens for larger grid size, with the main change occurring near $p \approx 0.59$.

3.2 Average burn duration and peak near the transition

Figures 2a and 2b show the average number of timesteps until the fire extinguishes. In both grid sizes, the burn duration increases as p approaches the transition region, reaches a peak

near $p \approx 0.60$, and then decreases again for larger p.

This peak occurs because, near the transition, the fire tends to propagate through large and irregular connected clusters, producing longer-lived dynamics before extinction. For low p, clusters are small and the fire dies out quickly. For very high p, although spanning is likely, the burning front can move more directly through the system, reducing the total duration compared with the near-critical regime. The peak is much larger for $N = 500$ than for $N = 100$, reflecting the larger spatial extent and longer propagation paths available in larger systems.



(a) $N = 100$, $M = 200$.

(b) $N = 500$, $M = 100$.

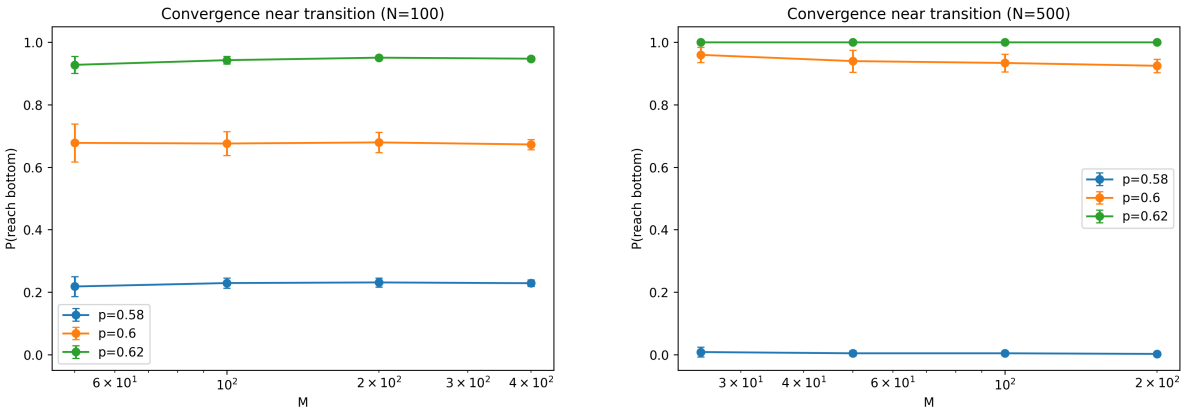
Figure 2: Average number of timesteps before the fire extinguishes as a function of tree density p. A pronounced peak is observed near the transition region, with a much larger peak for the larger grid.

3.3 Convergence with respect to the number of repeats M

The project asks whether $M = 50$ repeat runs is sufficient for convergence, and whether the answer depends on p. Figures 3a and 3b show convergence behaviour for probabilities near the transition ($p = 0.58, 0.60, 0.62$), where fluctuations are expected to be largest.

The main trend is that the estimated mean probability stabilises relatively quickly, but the confidence intervals shrink more slowly with increasing M. This means that $M = 50$ is often enough for a qualitative conclusion (e.g. identifying whether the system is mostly subcritical or mostly supercritical), but may not be sufficient for a precise estimate near the transition. In particular, the uncertainty is largest around $p = 0.60$, where the outcome varies most strongly from run to run.

Therefore, the answer to the convergence question *does depend on* p. Near the transition region, larger M (e.g. 100–200 or more) is desirable if tighter confidence intervals are required. Away from the transition, smaller M can be acceptable because the outcomes are closer to 0 or 1 and the variance is lower.



(a) $N = 100$.

(b) $N = 500$.

Figure 3: Convergence of the estimated bottom-reaching probability with increasing number of repeats M for probabilities near the transition. Error bars show 95% confidence intervals. The mean stabilises quickly, but uncertainty decreases more gradually, especially near $p = 0.60$.

3.4 Strong-scaling performance

Figure 4 shows the strong-scaling speedup for $N = 100$ and $N = 500$ at fixed $p = 0.6$, compared with ideal linear speedup. The larger grid ($N = 500$) achieves substantially better scaling than the smaller grid ($N = 100$), which is expected because the compute-to-communication ratio is higher for larger subdomains.

For $N = 100$, speedup improves initially but saturates and eventually degrades at high task counts. This indicates that communication and synchronisation overheads dominate once the local workload becomes too small. In contrast, for $N = 500$, speedup continues to increase up to 16 tasks, although it remains sublinear. The gap from ideal linear scaling can be attributed to halo exchanges at each timestep and the cost of two global `MPI_Allreduce` operations per timestep.

These results suggest that the choice of task count should depend on problem size. For smaller grids, using many tasks can be inefficient, whereas larger grids benefit more from parallel execution before overheads dominate.

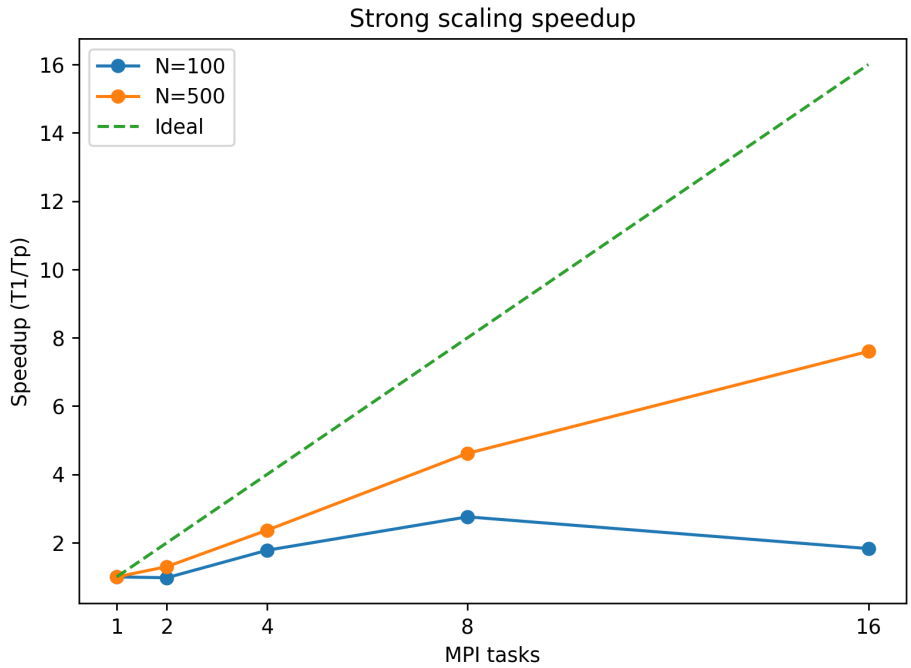


Figure 4: Strong-scaling speedup at $p = 0.6$ for two grid sizes. The dashed line shows ideal linear speedup. The larger grid ($N = 500$) scales substantially better than $N = 100$, while both remain sublinear due to communication and synchronisation overheads.

4 Discussion and limitations

The results are consistent with the expected qualitative behaviour of a forest fire / percolation-like model. The probability of reaching the bottom exhibits a threshold near $p \approx 0.59$, and the average number of steps peaks in the same region. The sharpening of the transition with increasing grid size supports the interpretation that finite-size effects smooth the transition for smaller systems.

The convergence analysis highlights an important practical point: the number of repeats M required depends on what quantity is being estimated and how much uncertainty is acceptable. A single rule such as "$M = 50$ is enough" is too simplistic. Instead, convergence should be judged relative to the uncertainty in the estimated observable, particularly near the transition where run-to-run variation is largest.

The performance results also show a clear trade-off between parallelism and overhead. The current implementation is robust and conceptually simple, but it performs halo exchange every timestep and requires global collective communication for termination and bottom-reach detection. These collective operations can become a bottleneck, especially for small problem sizes or high task counts.

A further limitation is that only a one-dimensional row-wise decomposition was used. While this is appropriate for the current grid sizes and project scope, more advanced decomposition strategies may improve scaling for larger problems.

5 Future work

There are several possible improvements that could extend this work:

- **Improved convergence analysis:** Use a formal stopping rule based on a target CI95 half-width (for example, stop increasing M once the half-width falls below a chosen tolerance).
- **Performance optimisation:** Reduce synchronisation overhead by combining global checks where possible or exploring alternative reduction strategies.
- **Communication/computation overlap:** Investigate non-blocking halo exchange and overlap with interior updates.
- **Larger grids / finite-size study:** Run additional grid sizes to study how the transition sharpness and peak burn duration scale with N.
- **Alternative neighbourhoods:** Compare von Neumann and Moore neighbourhoods to assess changes in threshold behaviour.

6 Conclusion

A parallel C++/MPI forest fire model was implemented and used to study both statistical behaviour and computational performance. The simulations show a clear transition in the probability of the fire reaching the bottom near $p \approx 0.59$, with a sharper transition for larger grid size. The average burn duration peaks near the transition, indicating longer-lived dynamics in the near-critical regime.

Convergence with respect to the number of repeat runs depends on p: $M = 50$ may be sufficient for qualitative conclusions, but larger M is needed near the transition to reduce uncertainty. Strong-scaling tests show that the implementation provides useful speedup for larger grids, while smaller grids saturate earlier due to communication and synchronisation overheads. Overall, the project demonstrates both the scientific behaviour of the model and the practical limits of MPI parallelisation for this class of cellular automaton.

Use of AI Tools

AI tools (including ChatGPT) were used as a writing grammar aid during this report. Code debugging process, were carried out.

Code and Supplementary Materials

The assessed C++ source code submitted for this project contains the main simulation implementation. For reproducibility and completeness, a supplementary GitHub repository is also provided, containing:

- the complete C++/MPI codebase,
- Python plotting scripts used to generate figures,

- Slurm/bash job scripts used for BluePebble runs,
- supplementary notes and workflow files.

Repository link: `https://github.com/GoJeonPa021/Year3_Mini_Project`

References

- SCIF30005 Forest Fire Mini Project specification and workshop materials.
- MPI documentation (for `MPI_Sendrecv` and `MPI_Allreduce`).